CSC490 A3

Zhuorui(Jack) Jiang 1008917059
Qixiang(David) Fan 1008847241
Zihan(Jason) Zhao 1010128274
Gabriel Won Bin Lee 1007488504

March 6, 2026

1 Part One: Architecture Review

1.1 Recent LLM Architecture Improvement Approaches

SwiGLU Activation

- *Motivation:* Standard FFNs use a single linear layer with a simple activation like ReLU. Smoother, non-monotonic activations like Swish often yield better gradient flow. Gated Linear Units (GLUs) enhance expressiveness by multiplying two linear projections (one passed through an activation), allowing dynamic modulation of information. SwiGLU merges these concepts by using a Swish activation in a GLU-style FFN, increasing nonlinearity and flexibility for the representations (Shazeer 2020).
- *Technical Detail:* Replace the standard FFN in the transformer block with:

$$\text{FFNSwiGLU}(x, W, V, W_2) = (\text{Swish}_1(xW) \otimes xV)W_2$$

- *Potential impact:* The increased expressivity and smoother optimization landscape of the FFN are expected to improve overall model output quality and convergence stability.

Parallel Layers

- *Motivation:* Standard Transformer blocks compute the MLP (FFN) after the attention output in a serialized manner, limiting opportunities for improved computational efficiency. Parallel layers allow the MLP and attention to operate independently on the same normalized input, enabling better utilization of matrix multiplications at large scales (Chowdhery et al. 2022).
- *Technical Detail:* Replace the standard serialized block:

$$y = x + \text{MLP}(\text{LayerNorm}(x + \text{Attention}(\text{LayerNorm}(x))))$$

  with a parallel formulation:

$$y = x + \text{MLP}(\text{LayerNorm}(x)) + \text{Attention}(\text{LayerNorm}(x))$$

  allowing MLP and attention computations to be fused.
- *Potential impact:* The parallel formulation reportedly speeds up training by roughly 15% at large scales. While ablation studies indicate negligible quality degradation for massive models (62B–540B parameters), a small degradation was observed for smaller models (8B parameters). The effects of this architecture change on nano/pico-scale models remain underexplored and merit further empirical investigation.

DeepSeek Sparse Attention

- *Motivation:* Standard Transformer self-attention scales quadratically ($O(L^2)$) with sequence length, becoming computationally infeasible for very long contexts. DeepSeek Sparse Attention (DSA) introduces a dynamic sparsity mechanism to reduce this cost while preserving performance in long-context tasks (DeepSeek-AI et al. 2025).
- *Technical Detail:* Instead of computing full dense attention over all token pairs, DSA dynamically selects a subset of the most relevant tokens per query (e.g., via a light indexer and top-k pipeline) and computes attention only on those selected tokens, significantly reducing attention matrix operations while retaining key dependencies.
- *Potential impact:* By replacing $O(L^2)$ dense attention with dynamic sparse computation, DSA substantially reduces computation and memory costs for long sequences. This enables efficient training and inference at scale with minimal quality loss, supporting the push toward longer context windows.

1.2 Nanochat Model Architecture Diagram

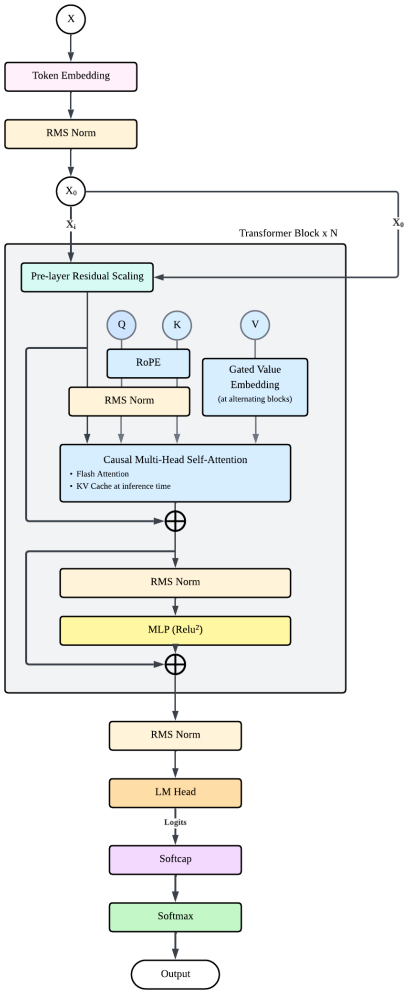

Figure 1: Nanochat model architecture

The architecture largely follows a standard decoder-only Transformer but includes several efficiency and stability modifications, as shown in Figure 1. It uses RoPE positional encoding instead of learned positional

embeddings and applies parameter-free RMSNorm throughout the network. Attention stability is improved with QK normalization, while ReLU^2 activations replace GELU for cheaper and sparser computation. The model uses untied token embeddings and output projection (lm_head) and applies logit soft-capping to bound output logits. To improve long-context efficiency, the model implements a sliding-window attention pattern (SSSL) that alternates three short-context layers (1024 tokens) with one long-context layer (2048 tokens), with the final layer attending to the full context. Additionally, the architecture introduces Value Embeddings, which inject token-dependent embeddings into the attention value vectors via a learned gate, greatly increasing parameter count with minimal additional compute. Each layer also includes learnable residual mixing scalars that combine the current hidden state with the initial embedding. For efficient attention computation, the model relies on FlashAttention-3, using a native (B, T, H, D) tensor layout and falling back to PyTorch SDPA when unsupported hardware is used.

2 Part Two: Ablations on a nano nanochat

2.1 Choice of PicoChat Configuration

The original nanochat configuration used a Transformer depth of 20 layers. To construct a smaller picochat model under limited computational credits, we reduce the depth to 12 layers while keeping other architectural hyperparameters fixed. In a small-batch autoregressive inference setting (device batch size ≤ 8), reducing the number of Transformer blocks improves time efficiency by lowering layer-wise memory access and matrix operations. Prior work on depth pruning shows that removing entire blocks reduces memory movement between activations and weights and yields actual inference speedups under small-batch conditions (Kim et al. 2024). This preserves the overall architecture while providing a computationally efficient experimental setting for ablation experiments. All other architectural and training configurations remain unchanged, forming the baseline picochat model used in subsequent comparisons.

2.2 Ablation Setup

To evaluate the impact of architectural modifications, we conduct controlled ablation experiments on the picochat configuration described above. The baseline model follows the original nanochat architecture with a ReLU^2 feed-forward network and a standard sequential Transformer block.
We evaluate three architectural variants:

- **Parallel**: We adopt a parallel residual formulation inspired by PaLM, where the attention and MLP modules operate on the same normalized input instead of sequentially.
- **SwiGLU**: We replace the standard feed-forward network with a SwiGLU activation module to increase the expressiveness of the Transformer MLP.
- **Parallel+SwiGLU**: We combine both modifications, applying the parallel residual formulation together with the SwiGLU feed-forward activation. This setting allows us to examine whether the benefits of parallelization and gated activations are complementary.

For each variant, we train a separate picochat model while keeping all other training and architectural configurations unchanged. This controlled setup ensures that observed performance differences can be attributed directly to the architectural changes.

2.3 Results

The ablation results in Table 1 reveal several consistent patterns across architectural variants. Rather than focusing solely on aggregate CORE scores, we analyze performance trends by task category to better understand the functional impact of each architectural modification.

1. Parallel Layers Improve Structured and Multi-Step Reasoning.

The Parallel configuration achieves the highest overall CORE score (0.1297), outperforming the baseline (0.1278). Notably, the gains are concentrated in structured reasoning and symbolic-style benchmarks such as CommonsenseQA (+9.6 points over baseline), Winogrande (+0.009), BIG-bench Dyck Languages (+0.038), BIG-bench Operators (+0.029), and AGI-Eval LSAT Analytical Reasoning (+0.048). These tasks require either multi-step inference, rule tracking, or compositional reasoning.

The parallel residual formulation alters the information flow within each transformer block. Instead of sequentially conditioning the MLP on the attention output, both modules operate on the same normalized input. This effectively reduces intra-layer dependency depth and may stabilize gradient propagation in smaller models.

Importantly, the improvement does not come at the expense of degradation in most language modeling tasks. This suggests that parallelization primarily improves representation integration rather than altering fundamental language modeling capability.

2. SwiGLU Shows Selective Gains but Lower Global Stability.

The SwiGLU variant achieves a lower overall CORE score (0.1203), despite outperforming the baseline on specific benchmarks such as OpenBookQA (+0.018), BIG-bench Dyck Languages (+0.030), and modest improvements in certain generative tasks. However, it exhibits noticeable degradation on tasks like BoolQ (-0.070 relative to baseline) and ARC Challenge (-0.013).

SwiGLU increases the expressivity of the feed-forward network by introducing multiplicative gating. While this theoretically enhances nonlinear representation capacity, it also increases effective parameter interaction complexity. In smaller models, this added flexibility may lead to overfitting or less stable optimization, especially when training data and model width are limited. The reduced CORE score suggests that additional expressivity does not automatically translate into better generalization at this scale.

3. Parallel+SwiGLU: Complementary but Not Additive.

The combined Parallel+SwiGLU model achieves a CORE score of 0.1231, outperforming SwiGLU alone but not surpassing Parallel. Interestingly, it produces the strongest results on several individual tasks, including Jeopardy QA, PIQA, AGI-Eval LSAT Analytical Reasoning, SQuAD, and BIG-bench Language Identification.

This indicates partial complementarity: parallelization appears to stabilize optimization, allowing the gated activation to contribute meaningfully on certain reasoning or knowledge-retrieval benchmarks. However, the improvements are task-specific rather than uniform, and aggregate performance does not exceed the Parallel-only configuration. This suggests that at small model scale, architectural benefits are not strictly additive. Instead, there appears to be an interaction effect where optimization dynamics dominate over raw expressivity gains.

4. Training Efficiency and Optimization Dynamics.

Figures 2 and 3 provide additional insight. Baseline and SwiGLU exhibit nearly identical training time and throughput, indicating that the added gating does not significantly alter computational cost at this scale. In contrast, Parallel and Parallel+SwiGLU demonstrate consistently higher throughput and lower accumulated training time.

This confirms that parallelizing attention and MLP reduces sequential dependency overhead and improves hardware utilization. Crucially, the architectural modification that improves efficiency (Parallel) is also the one that improves aggregate performance. This alignment between optimization efficiency and benchmark accuracy strengthens the conclusion that the parallel residual formulation is particularly well-suited for pico-scale models.

6. Overall Interpretation and Comparison to Prior Work.

Across all experiments, the parallel residual architecture emerges as the most robust and reliable modification at this model scale. It improves reasoning performance, maintains or slightly improves language modeling benchmarks, and increases training efficiency.

Notably, this finding contrasts with observations from (Chowdhery et al. 2022), who report that parallel residual layers may lead to performance degradation at smaller model scales. In our experiments, however, parallel residual layers not only improved training throughput and reduced accumulated training time, but also yielded consistent gains in aggregate benchmark performance (CORE). This suggests that, at pico-scale, the optimization benefits introduced by parallelization may outweigh any potential representational drawbacks.

One possible explanation is that smaller models are more sensitive to gradient flow and optimization stability than to marginal increases in expressivity. By reducing sequential dependency within transformer blocks, the parallel formulation may improve conditioning and hardware utilization in a way that disproportionately benefits low-capacity regimes.

These findings highlight that architectural effects are scale-dependent, and that conclusions drawn at very large parameter counts do not always extrapolate directly to smaller regimes.

Task	Baseline	Parallel	SwiGLU	Parallel+SwiGLU
HellaSwag (Zero-shot)	**0.328122**	0.324039	0.323641	0.322645
Jeopardy QA	0.006141	0.003779	0.003307	**0.010864**
BIG-bench QA (Wikidata)	**0.321293**	0.309532	0.308449	0.305989
AI2 ARC Easy	0.512205	**0.518098**	0.513889	0.512205
AI2 ARC Challenge	**0.261945**	0.255973	0.249147	0.251706
Choice of Plausible Alternatives (COPA)	**0.600000**	0.560000	0.580000	0.540000
CommonsenseQA	0.231777	**0.327600**	0.281736	0.321867
Physical Interaction QA (PIQA)	0.631665	0.625680	0.620783	**0.633297**
OpenBookQA	0.298000	0.292000	**0.316000**	0.292000
LAMBADA (OpenAI)	0.270134	0.272657	0.264118	**0.272851**
HellaSwag (Full)	**0.322446**	0.322247	0.320454	0.318662
Winograd Schema Challenge	0.575092	0.571429	0.567766	**0.578755**
Winogrande	0.512234	**0.521705**	0.517758	0.507498
BIG-bench Dyck Languages	0.084000	**0.122000**	0.114000	0.084000
AGI-Eval LSAT Analytical Reasoning	0.234783	0.282609	0.282609	**0.291304**
BIG-bench CS Algorithms	**0.403788**	**0.403788**	0.395455	0.363636
BIG-bench Operators	0.085714	**0.114286**	0.066667	0.066667
BIG-bench Repeat Copy Logic	**0.031250**	0.000000	0.000000	0.000000
SQuAD	0.082971	0.094607	0.085903	**0.097351**
CoQA (Conversational QA)	0.111111	**0.132156**	0.121132	0.127270
BoolQ	**0.575841**	0.538532	0.506116	0.550765
BIG-bench Language Identification	0.254800	0.252800	0.252900	**0.255200**
CORE (Mean Centered Accuracy)	0.127812	**0.129694**	0.120288	0.123133

Table 1: Accuracy and CORE comparison across model settings.

Figure 2: Total training time across training steps for the Baseline, Parallel, SwiGLU, and Parallel+SwiGLU configuration models.

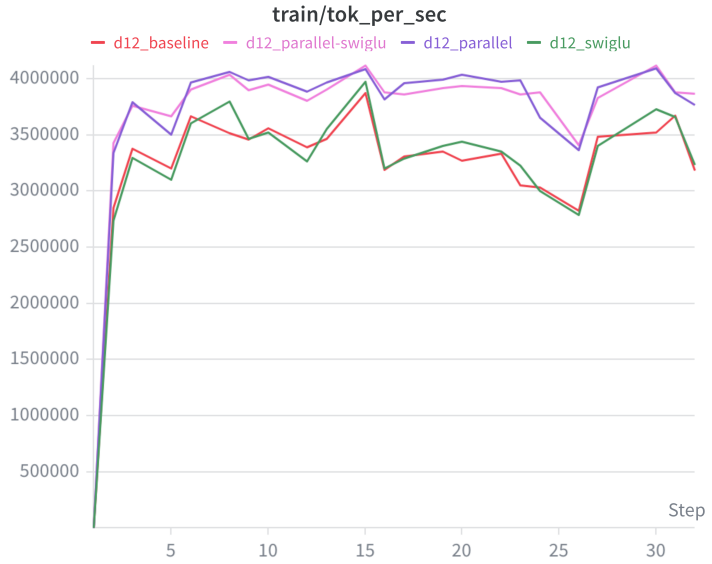

Figure 3: Training throughput measured in tokens per second for the four model variants.

2.4 Scaling Implications

Our ablation results are obtained using the smaller picochat configuration, which allows rapid experimentation under limited computational resources. Although the absolute performance values may differ for larger models, the relative trends observed in our experiments are likely to generalize.

In particular, the parallel transformer block improves the overall CORE score while maintaining comparable training efficiency, suggesting that this architectural modification may remain beneficial when scaling to

larger models. In contrast, the SwiGLU modification does not improve overall benchmark performance in our experiments and introduces additional computational overhead during training. While SwiGLU has been shown to be effective in larger models in prior work, our results suggest that its benefits may depend on model scale and training configuration.

2.5 Training Cost and Resource Usage

Due to limited computational credits, we carefully designed our experiments to balance exploration of architectural changes with efficient resource usage. All ablation experiments were conducted using the smaller picochat configuration rather than the original nanochat model. This reduced the computational cost of each training run while still allowing us to observe meaningful differences between architectural variants.

Each model variant (Baseline, Parallel, and SwiGLU) was trained separately while keeping all other training settings unchanged. Using a smaller model significantly reduced GPU training time and enabled multiple experiments to be completed within the available credit budget. This approach allowed us to perform controlled ablation studies without requiring the substantially higher cost associated with training larger models.

Although larger models may produce different absolute performance results, using the picochat configuration provides an efficient way to study the relative impact of architectural modifications before scaling them to larger models.

Model Variant	**Hardware**	**Duration (min)**	**Est. Cost**
Baseline	8×H100	4.65	$2.45
Parallel	8×H100	4.23	$2.23
SwiGLU	8×H100	5.14	$2.71
Parallel+SwiGLU	8×H100	4.31	$2.27
Total		**18.33**	**$9.65**

Table 2: Training time and computational cost breakdown

3 Part Three: Extending the context window

To optimize our compute budget while fulfilling the context window requirements, we implement a two-stage, length-incremental training strategy. We initialize the first pre-training phase by reducing the default nanochat sequence length from 2,048 to 512 tokens. Because the standard self-attention mechanism computes a dense Gram matrix that scales quadratically with respect to sequence length $O(L^2)$, this aggressive reduction decreases the memory footprint and per-step attention FLOPs by a factor of approximately 16. This highly efficient regime allows the picochat baseline to rapidly converge on foundational syntactic structures and local token dependencies over a subset of the pre-training dataset, preventing early compute waste.

Once local representations are stabilized, we utilize this reduced-sequence checkpoint to initialize the second phase, expanding the sequence length back to the target 2,048 tokens and continuing training. Training at this extended length allows the model to calibrate its positional embeddings for long-range dependencies without the prohibitive computational cost of full-context training from initialization.

3.1 Configurations for picochat and Justification

We conduct our context extension experiments using a scaled-down "picochat" configuration to remain strictly within compute constraints while maintaining sufficient representational capacity for meaningful ablation analysis. Specifically, our baseline picochat architecture is d12, defined by a depth of 12 transformer layers, 768 channels, 12 heads with 128 head dim, yielding approximately 162 million non-embedding parameters. This reduced configuration provides a computationally lightweight, experimentation-grade baseline

that is highly sensitive to hyperparameter and architectural shifts. Consequently, the effects of sequence length modulation manifest clearly during evaluation rather than being obscured or compressed by the over-parameterization typical of larger models.

The choice to initialize the first pre-training phase with a reduced context window of 512 tokens is directly motivated by the *Shortformer* methodology introduced by (Press, Smith, and Lewis 2021). Their research demonstrates that exposing a model to excessively long sequences from random initialization can be actively detrimental to early-stage convergence. By restricting the initial context window to 512 tokens, the model is constrained to efficiently capture fundamental local syntactic structures and short-range dependencies without being overwhelmed by the complex noise of long-distance token interactions.

Furthermore, Press et al. establish that this staged curriculum approach—training on short subsequences before incrementally extending to longer ones—not only accelerates initial parameter updates but surprisingly yields substantial improvements in final model perplexity. Thus, the 512-token initialization serves as both a compute-saving measure and a deliberate curriculum learning mechanism. It ensures the model mitigates the "early token curse" and solidifies its local attention patterns before we introduce the computational burden and structural complexity of the full 2,048-token context window in the second phase of training.

To operationalize this two-stage curriculum, we partition our pre-training dataset, allocating 80% of the total training tokens to the initial 512-token phase and the remaining 20% to the extended 2,048-token phase. This highly skewed distribution is empirically justified by dataset distribution analyses, such as the findings presented by Vries (2023) in "In the long (context) run." Their analysis of massive pre-training corpora, including CommonCrawl and GitHub, reveals that approximately 80% of tokens belong to documents that naturally fit within standard, shorter context windows. Consequently, only the remaining 20% of tokens derive from documents of sufficient length to meaningfully benefit from extended context training. Forcing the model to process the initial 80% of the corpus through a massive context window is computationally wasteful. By dedicating only the final 20% of the dataset to the extended 2,048-token window, we provide sufficient exposure for the model to recalibrate its positional embeddings and develop long-range attention routing on the specific data that actually requires it, thereby maximizing compute efficiency without sacrificing capabilities.

3.2 Experiment Steps

To rigorously evaluate the effectiveness of our length-incremental curriculum, we operationalize the two-stage training procedure as follows.

Stage 1: Short-Context Pre-training (512 Tokens).

We initialize a picochat d12 model with a maximum sequence length of 512 tokens and train it for 2,205 optimization steps. The training budget is calibrated to maintain the default parameter-to-data ratio of approximately 10.5, ensuring that the model receives a comparable effective number of token updates relative to the standard nanochat configuration.

During this phase, the model is exposed exclusively to the first 80% of the shuffled pre-training corpus, with all documents segmented into contiguous 512-token blocks. This design enforces a strictly local attention regime, encouraging the model to converge on syntactic regularities, short-range compositional patterns, and stable token-level representations before encountering longer-range dependencies.

At the conclusion of Stage 1, we checkpoint the model parameters, optimizer states, and learning rate scheduler to preserve both representational and optimization continuity.

Stage 2: Context Expansion and Long-Range Calibration (2,048 Tokens).

We then resume training from the Stage 1 checkpoint, expanding the maximum context window from 512

to 2,048 tokens. Importantly, no weights are reinitialized during this transition; positional embeddings are extended (or reparameterized, depending on implementation) to accommodate the larger sequence length while preserving previously learned positional structure.

The model is subsequently trained for an additional 500 optimization steps using the remaining 20% of the pre-training dataset. In this phase, documents are segmented into 2,048-token sequences, allowing the model to recalibrate attention patterns for long-range interactions and global dependency modeling. Because the majority of foundational language structure has already been internalized during Stage 1, this second phase functions primarily as a refinement stage, adjusting positional encoding dynamics and long-distance routing behavior rather than relearning basic syntax.

Training Continuity and Optimization Considerations.

Crucially, we retain identical optimizer hyperparameters, learning rate schedule, and batch size across both stages to isolate the effect of sequence length modification. By preserving optimization dynamics, we ensure that observed performance differences arise from curriculum structure rather than confounding hyperparameter shifts.

The asymmetry in step allocation (2,205 vs. 500 steps) reflects the token distribution characteristics of the underlying corpus, as discussed previously. Since the majority of tokens originate from shorter documents, extended-context exposure is treated as a targeted specialization phase rather than a primary learning regime.

This two-stage protocol allows us to decouple the acquisition of local linguistic structure from long-range positional calibration, providing a controlled experimental framework for assessing whether length-incremental training improves compute efficiency and final downstream performance under strict resource constraints.

3.3 Choice of Evaluation

Length-Stratified Perplexity (BPB)

Our primary quantitative metric is validation Bits-Per-Byte (BPB), evaluated at sequence lengths of 512, 1024, and 2048 tokens. BPB directly measures token-level predictive performance while remaining comparable across varying context sizes. Evaluating at multiple lengths is essential for disentangling three distinct behaviors: (1) short-context competence, (2) robustness under context scaling, and (3) long-context calibration.

Because our central hypothesis concerns length generalization, single-length evaluation would be insufficient. A model trained exclusively at 512 tokens may appear competitive when evaluated at 512 yet degrade substantially at 2048 due to distributional mismatch. By stratifying evaluation across increasing sequence lengths, we explicitly measure how performance shifts as the effective receptive field expands.

CORE: Structural Coherence Metric

While BPB captures token-level likelihood, it does not directly measure higher-order discourse structure. To complement perplexity, we report the CORE metric, which quantifies global coherence and structural consistency across generated sequences. This metric is particularly relevant in long-context experiments, where improvements may manifest not only as marginal likelihood gains but also as more stable paragraph organization and reduced degenerative repetition.

Including CORE ensures that improvements are not merely artifacts of local token calibration but reflect genuine structural refinement in attention routing and positional encoding adaptation.

Conditioned and Unconditioned Generation

Beyond numerical metrics, we conduct qualitative evaluation under both conditioned and unconditioned generation settings. Conditioned prompts test specific competencies—including logical continuation, factual recall, and resistance to repetitive collapse—allowing us to assess whether context expansion alters short-range reasoning or knowledge retention.

Unconditioned generation, by contrast, provides a more sensitive probe of global discourse planning. Without prompt constraints, deficiencies in long-range coherence and thematic persistence become immediately visible. This setting is particularly diagnostic for detecting improvements in global structure that may not be fully captured by perplexity metrics alone.

Isolation of Curriculum Effects

All evaluations are performed under identical decoding settings and without additional fine-tuning, ensuring that differences arise solely from the training curriculum. By holding optimizer hyperparameters, data distribution, and architectural configuration constant, we isolate sequence length as the only experimental variable.

3.4 Result and Evaluation

Quantitative Evaluation

Table 3 presents validation BPB across evaluation sequence lengths and the CORE metric for both the 512-only baseline and the extended 512→2048 model.

Across all evaluation lengths, the length-incremental model consistently outperforms the 512-only baseline. At 512 tokens, BPB improves from 0.899209 to 0.895385 ($\Delta = -0.0038$). More notably, the improvement increases as the evaluation length grows: at 1024 tokens, BPB decreases by 0.0120, and at 2048 tokens, by 0.0176. This monotonic widening of the performance gap indicates that the second training stage successfully enhances long-context modeling without sacrificing short-context competence.

Importantly, short-context performance does not degrade after context expansion. The extended model achieves better BPB even when evaluated at 512 tokens. This suggests that long-context calibration improves global representation quality rather than inducing specialization at the expense of local modeling.

The CORE metric further corroborates this finding, increasing from 0.1185 to 0.1246. Because CORE measures coherence and structural consistency, this gain indicates that improvements are not limited to token-level likelihood but extend to higher-order sequence organization.

Length Generalization Behavior

The 512-only model exhibits clear degradation as evaluation length increases ($0.8992 \rightarrow 0.9166$). In contrast, the extended model maintains much flatter degradation ($0.8954 \rightarrow 0.8990$). This demonstrates that models trained exclusively on short contexts suffer from distributional shift when evaluated on longer sequences, whereas staged context expansion mitigates this mismatch.

The widening BPB gap at 2048 tokens strongly suggests that exposure to long sequences recalibrates positional representations and attention routing patterns in a way that generalizes across scales. Crucially, this improvement is achieved with only 20% of total training tokens allocated to the long-context phase, indicating high data efficiency.

Qualitative Analysis: Conditioned Generation

Table 4 provides conditioned completions. Several trends emerge:

- **Reduced Degenerate Repetition:** The 512-only model frequently repeats phrases (e.g., "The opposite of hot is cold" repeated verbatim). The extended model exhibits more alternation and structural variation.
- **Improved Logical Continuation:** In the temporal reasoning example ("If yesterday was Friday. . . "), the extended model maintains correct weekday progression across multiple clauses, while the baseline begins to repeat templated patterns.
- **Factual Recall Stability:** Both models correctly list planets and identify Paris as the capital of France, indicating that short-context knowledge acquisition is preserved after expansion.
- **Arithmetic Limitation:** Both models fail to solve the simple linear equation, suggesting that improvements stem primarily from sequence modeling rather than symbolic reasoning gains.

Overall, the extended model exhibits slightly improved local coherence and reduced repetitive collapse, though improvements in short, fact-based prompts remain subtle.

Qualitative Analysis: Unconditioned Generation

In unconditioned samples (Table 5), differences are more pronounced:

- The 512-only model often drifts semantically and introduces incoherent structural transitions.
- The extended model maintains longer thematic consistency before divergence.
- Paragraph structure appears more stable in the extended model, suggesting improved global planning.

Although both models still exhibit mid-scale incoherence (expected given the 162M parameter scale), the extended model demonstrates noticeably improved discourse continuity.

Key Findings

Contrary to concerns that context expansion may degrade performance in smaller models, our results demonstrate the opposite trend: the staged 512→2048 curriculum improves both short- and long-context BPB while increasing structural coherence. The gains are largest at longer evaluation lengths, confirming that the second phase effectively calibrates long-range dependencies without destabilizing previously learned local representations.

These findings suggest that, even under strict compute constraints, length-incremental training provides a compute-efficient pathway to improved long-context generalization without performance regression on short contexts.

Evaluation Seq Len	**512-only**	**512→2048**
512	0.899209	**0.895385**
1024	0.906552	**0.894559**
2048	0.916570	**0.898969**
CORE Metric	0.1185	0.1246

Table 3: Validation BPB across evaluation sequence lengths and CORE metric for the 512-only model (step 2205) and the extended 512→2048 model (step 2705). Lower BPB is better; higher CORE is better.

Table 4: Conditioned text samples from the 512-only and extended models.

Prompt	**512-only**	**512→2048**
The capital of France is	The capital of France is Paris, and the city is the capital of the French Republic. The city is	The capital of France is Paris, and the city is the capital of the French Republic. The city is
The chemical symbol of gold is	The chemical symbol of gold is the symbol of the Roman Empire. The symbol of gold is the symbol of the	The chemical symbol of gold is the symbol of the metal. It is a symbol of the metal and is used
If yesterday was Friday, then tomorrow will be	If yesterday was Friday, then tomorrow will be Saturday. If yesterday was Sunday, then tomorrow will be Monday. If yesterday was	If yesterday was Friday, then tomorrow will be Saturday. If yesterday was Saturday, then tomorrow will be Sunday. If yesterday was
The opposite of hot is	The opposite of hot is cold. The opposite of hot is cold. The opposite of cold is hot.	The opposite of hot is cold. The opposite of cold is hot. The opposite of hot is cold.
The planets of the solar system are:	The planets of the solar system are: Mercury, Venus, Earth, Mars, Jupiter, Saturn, Uranus, Neptune	The planets of the solar system are: Mercury, Venus, Earth, Mars, Jupiter, Saturn, Uranus, Neptune
My favorite color is	My favorite color is red. I love it because it is so easy to make. It is also	My favorite color is red. I love it because it is the color of the blood. It is
If 5*x + 3 = 13, then x is	If 5*x + 3 = 13, then x is 5*x + 3 = 13. If 5*x	If 5*x + 3 = 13, then x is 5*x + 3 = 13. If 5*x

Table 5: Unconditioned generation samples from the 512-only and extended models.

Model	Unconditioned Generated Samples
512-only	The Summary for Policymakers (SMPs) Guidance & Updates for 2020 The Statement and Context for School Leadership "School leaders should begin to keep school system doors locked and doors unlocked." The School Leadership Conference was first scheduled to begin on Monday, November 29, 2021. It took roughly two and a half weeks for the leaders to attend. The governing board voted to authorize one research memo, called 'Fakery: The School Leaders" for the upcoming school year 2023. Participants will be allowed to participate in this annual publication. Blended Learning by
	SAMBURS of scientific questions are from all corners of the world. What if humans — we are here — are sucking up drifting chunks of iron 100 feet into the Arctic by now? How much of the mining's impact on life and environment will be in sea ice and soils? The universe has many wondered about this, how will humanity possibly have impacted? Drilling over the Arctic is getting closer and closer. Four clamps with a probe remain in position, each about 224 meters down in a clockwise direction, with half a meter of drill weight to begin with. So to get a clear plume, the
512→2048	The Summary of Working With Children (Advanced, 2018) If you are a teacher, you should be able to understand what problems your elementary students are working through. KTN teachers are losing faith in the turn-the-ideal teaching. Younger students love being asked questions and assessments and hesitate to be overwhelmed with a teacher who constantly repeats boring and extenuating assignments and sometimes repeats repetitive questions or problems. We add to that one research done in Finland on 121 kindergarten juniors students from grades nine to 10 interviewed by 2011 journalists in collaboration with the Finnish Broadcasting Corporation
	SANE 240 EXAMPLE (Situation) Wordle-self quiz — locale collisionsthe origin of the custom, the explanation, Tafo',haintily after liveshop, Tat' Tisf-green sea-tides, the Camel Corona... The dictionary definition of the word "cu'fo' is uncertainty because of the lack at which it is rendered with every example.e.gographers, which dit to demonstrate, Why?, Rise,us, believe in, then to 'color the sources with white ones to clearly show the author's biases

4 Part Four: Training Your Final Nanochat

4.1 Experiments on other changes

For the final configuration of our nanochat model, we elected to proceed with the baseline architecture. To justify this choice, it is important to note that we conducted several exploratory architecture changes extending beyond the ablations performed in Parts 2 and 3. Specifically, we implemented and evaluated three advanced attention mechanisms to see if they would yield better performance:

- **Differential Attention:** This mechanism computes attention scores by taking the difference between two separate softmax attention maps. The theoretical motivation is to cancel out attention noise and more strongly highlight the most salient features within the context. (Ye et al. 2025)
- **Grouped Query Attention (GQA):** GQA acts as an interpolation between standard Multi-Head Attention (MHA) and Multi-Query Attention (MQA). It divides the query heads into groups, where each group shares a single key and value head. This is designed to reduce memory bandwidth and accelerate inference times while attempting to maintain the performance quality of MHA.(Ainslie et al. 2023)

- **Per-Head Gated Attention:** This approach introduces a learnable gating mechanism (a scalar or vector) for each attention head. It dynamically scales the output of each head, allowing the model to theoretically suppress less informative heads and route information more efficiently across the network layers.(Qiu et al. 2025)

Despite their theoretical advantages and documented successes in larger-scale models, all three of these architectural modifications resulted in a worse core evaluation score compared to the baseline nanochat model when trained in our specific environment. Because these changes introduced additional complexity without a commensurate gain in evaluation performance, our final configuration relies on the previous proven architecture changes that have positive impact on the final evaluation.

Table 6: Ablation Results for Nanochat Architecture Variants

Task	**Baseline**			**Differential**			**Gate**			**GQA**		
	Acc	Cent	T/s	Acc	Cent	T/s	Acc	Cent	T/s	Acc	Cent	T/s
hellaswag_zeroshot	**0.3267**	**0.1023**	**221092**	0.3153	0.0870	166474	0.3254	0.1006	206821	0.3078	0.0771	153822
jeopardy	**0.0066**	**0.0066**	1417	0.0038	0.0038	1200	0.0028	0.0028	**1510**	0.0024	0.0024	1102
bigbench_qa_wikidata	**0.3244**	**0.3244**	646	0.2450	0.2450	551	0.2899	0.2899	**674**	0.3005	0.3005	468
arc_easy	0.5198	0.3597	**708640**	0.4924	0.3232	549062	**0.5345**	**0.3793**	648847	0.4823	0.3098	475357
arc_challenge	0.2534	0.0046	**857385**	0.2509	0.0011	674045	0.2398	-0.0137	764612	**0.2619**	**0.0159**	597019
copa	**0.6000**	**0.2000**	**14980**	0.5900	0.1800	12655	0.5600	0.1200	14501	0.5300	0.0600	11198
commonsense_qa	0.2588	0.0735	**1008449**	0.2752	0.0940	788663	0.3276	0.1595	959571	**0.3612**	**0.2015**	698919
piqa	**0.6328**	**0.2655**	**431189**	0.6273	0.2546	333840	0.6317	0.2633	402805	0.6192	0.2383	291150
openbook_qa	**0.3160**	**0.0880**	**38756**	0.2980	0.0640	29663	0.3040	0.0720	36921	0.3060	0.0747	27793
lambada_openai	0.2649	0.2649	**976**	0.2548	0.2548	745	**0.2835**	**0.2835**	905	0.2633	0.2633	672
hellaswag	0.3177	0.0902	**1637250**	0.3121	0.0828	1096342	**0.3220**	**0.0961**	1569221	0.3077	0.0769	1148501
winograd	0.5714	0.1429	**28204**	0.5604	0.1209	21405	**0.5751**	**0.1502**	27713	0.5495	0.0989	19563
winogrande	0.4964	-0.0071	**33706**	0.5264	0.0529	24464	**0.5296**	**0.0592**	30643	0.5004	0.0008	23134
bigbench_dyck_languages	0.1110	0.1110	**1231**	0.0450	0.0450	943	**0.1320**	**0.1320**	1151	0.1110	0.1110	847
agi_eval_lsat_ar	0.2435	0.0543	**1728945**	**0.2652**	**0.0815**	1118789	0.2478	0.0598	1631921	0.2391	0.0489	1227800
bigbench_cs_algorithms	**0.4341**	**0.4341**	**1323**	0.3871	0.3871	1033	0.4265	0.4265	1245	0.3864	0.3864	935
bigbench_operators	0.0952	0.0952	**1160**	0.0762	0.0762	922	0.0905	0.0905	1097	**0.1095**	**0.1095**	815
bigbench_repeat_copy_logic	0.0000	0.0000	**7529**	0.0000	0.0000	6206	0.0000	0.0000	7389	**0.0313**	**0.0313**	5765
squad	**0.0781**	**0.0781**	**2140**	0.0279	0.0279	1279	0.0709	0.0709	2008	0.0357	0.0357	1468
coqa	**0.1195**	**0.1195**	**2016**	0.0973	0.0973	1541	0.1164	0.1164	1864	0.0904	0.0904	1394
boolq	0.5578	-0.1637	**1485729**	**0.5817**	**-0.1009**	815949	0.5040	-0.3053	1403059	0.4850	-0.3552	1013563
bigbench_language_id	0.2543	0.1796	**2526165**	**0.2548**	**0.1802**	914239	0.2499	0.1748	2431936	0.2546	0.1800	1830140
CORE	–	**0.1284**	–	–	0.1163	–	–	0.1240	–	–	0.1072	–

4.2 Configuration and Justification

For the final configuration of our model, we adopted the baseline architecture scaled to a depth of 24 layers, with one critical architectural modification: the integration of parallel transformer layers.

Originally introduced in the PaLM architecture (Chowdhery et al. 2022), parallel layers modify the standard sequential execution of the attention and feed-forward (MLP) blocks. In a standard transformer, these blocks are computed sequentially:

$$y_{seq} = x + \text{Attention}(\text{LN}(x))$$

$$z_{seq} = y_{seq} + \text{MLP}(\text{LN}(y_{seq}))$$

In contrast, our configuration computes the attention and MLP blocks simultaneously from the same layer-normalized input:

$$z_{parallel} = x + \text{Attention}(\text{LN}(x)) + \text{MLP}(\text{LN}(x))$$

This has been justifies in Part 2 of the assignment that have a overall positive impact on the final evalutation using the picochat version with depth =12.

Training Hyperparameters and Setup

The final model was trained using an 8-GPU distributed data parallel setup. The key training parameters were configured as follows:

- **Network Depth:** 24 layers, providing a sufficiently deep network to capture complex representations while remaining computationally tractable within our budget.
- **Batch Size:** A per-device batch size of 16 across 8 GPUs, resulting in an effective global batch size of 128.
- **Data-to-Parameter Ratio:** We targeted a training duration governed by a data-to-parameter ratio of 10.5, aligning our training token count with optimal scaling principles for the model's capacity. This is the default value for the vanilla nanochat model.
- **Evaluation Frequency:** To monitor performance without introducing excessive validation overhead, the core evaluation metrics were computed every 2000 steps.

This configuration struck the best balance between architectural efficiency, available training compute, and reliable downstream evaluation performance.

4.3 Scaling Law Prediction

In the following sections, we compare the final 24-layers nanochat model with parallel residual layers (trained as described in Section 4.2) against the baseline 12-layers picochat model.
To estimate the expected performance improvement when scaling from picochat to nanochat, we use the empirical scaling law proposed by Kaplan et al. (2020). The paper models language modeling loss as a power-law function of the number of model parameters:

$$L(N) = \left(\frac{N_c}{N}\right)^{\alpha} \tag{1}$$

where N denotes the number of parameters, N_c is a constant related to the dataset scale, and α is the scaling exponent. Following the values reported in the original work, we use $\alpha = 0.076$ and $N_c = 8.8 \times 10^{13}$.
For our models:

$$N_{\text{pico}} = 2.86262 \times 10^8 \tag{2}$$

$$N_{\text{nano}} = 1.384 \times 10^9 \tag{3}$$

The expected loss ratio between the two models can therefore be estimated as:

$$\frac{L(N_{\text{nano}})}{L(N_{\text{pico}})} = \left(\frac{N_{\text{pico}}}{N_{\text{nano}}}\right)^{\alpha} = \left(\frac{2.86262 \times 10^8}{1.384 \times 10^9}\right)^{0.076} \approx 0.888 \tag{4}$$

Assuming bits-per-byte (BPB) scales proportionally with language modeling loss, the predicted BPB for nanochat is:

$$\text{BPB}_{\text{nano,pred}} = 0.903230 \times 0.888 \approx 0.802 \tag{5}$$

Thus, the scaling law predicts that increasing the model size from 286M to 1.384B parameters should reduce BPB from approximately 0.903 to about 0.80.

4.4 Result and Evaluation

Table 7 compares the picochat baseline with the final nanochat model trained using the parallel residual architecture.

Model	Parameters	BPB (Eval)	CORE	Tokens/sec
Picochat (d=12)	286M	0.9032	0.1278	3.77M
Nanochat (d=24, Parallel)	1.384B	**0.7537**	**0.2496**	0.83M

Table 7: Comparison between picochat and the final nanochat model.

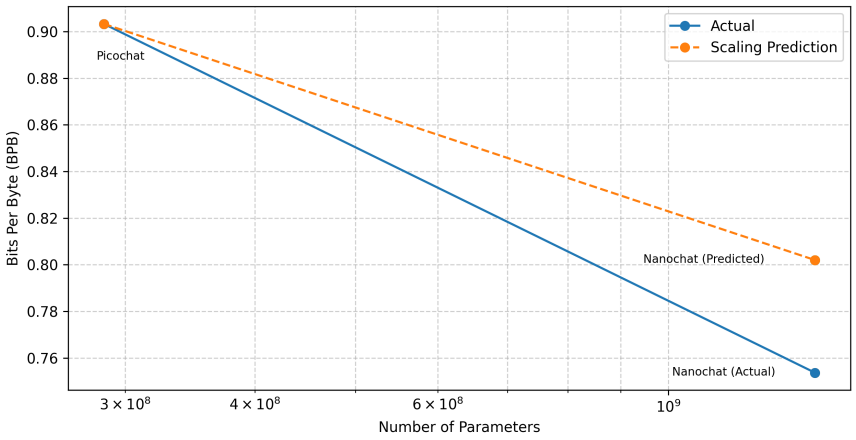

Figure 4: Scaling law prediction compared with actual performance. The dashed line shows the BPB predicted by the scaling law of Kaplan et al. (2020), while the solid line shows the empirically observed results for picochat and the final nanochat model.

Figure 4 compares the predicted scaling trend with the empirically observed results. The dashed line represents the BPB predicted by the scaling law, while the solid line shows the actual performance of the trained models.

The nanochat model achieves a BPB of 0.7537, outperforming the predicted value of approximately 0.802. This indicates that the performance gain cannot be explained solely by parameter scaling. A major contributing factor is the architectural modification: the parallel residual transformer allows the attention and feed-forward modules to operate on the same normalized input, reducing sequential dependency within each block. This improves gradient flow, optimization stability, and hardware utilization.

Scaling from 12 to 24 layers while retaining the parallel formulation compounds these benefits. The nanochat model shows a substantial reduction in language modeling loss (BPB: 0.903 $\rightarrow$ 0.754) and nearly doubles benchmark performance (CORE: 0.128 $\rightarrow$ 0.250), highlighting stronger multi-step reasoning and knowledge retrieval capabilities.

Additionally, the parallel residual architecture appears to improve training efficiency. Karpathy (2024) reports that a baseline d24 nanochat model trains in roughly 3.04 hours (10,949 sec) on 8×H100 GPUs using a parameter–data ratio of 12.0. In our experiment, the d24 nanochat model with parallel residual layers completes training in 9,373 seconds (2.6 hours) under similar hardware conditions and a default ratio of 10.5. While not a perfectly controlled comparison, this suggests that parallel layers may also contribute to faster token throughput and better hardware utilization, consistent with our Part 2 ablation study.

4.5 Emergent Abilities

The examples below illustrate several emergent abilities that appear when scaling the model from picochat (depth=12) to nanochat (depth=24, with Parallel layers). Although both models share the same training data and architecture family, the larger nanochat model consistently demonstrates stronger factual recall, more reliable arithmetic reasoning, and the ability to produce concise structured outputs such as code snippets.

Across multiple tasks, picochat frequently generates long but incoherent explanations, hallucinated facts, or incorrect reasoning steps. In contrast, nanochat is more likely to produce a direct and correct answer. This behavior can be observed in simple arithmetic problems (Questions 3–4), factual knowledge retrieval (Questions 6–8), grammatical knowledge (Question 9), and short programming tasks (Question 5).

Another noticeable difference is answer formatting. Picochat often produces repetitive or verbose text that drifts away from the question, whereas nanochat tends to generate concise responses that more closely match the expected format (e.g., a single number, word, or code snippet). This suggests improved internal representation of task structure and better instruction following.

Overall, these results indicate that increasing model depth from 12 to 24 layers leads to qualitative improvements in several capabilities, including basic reasoning, factual knowledge retrieval, and structured output generation. While the tasks remain relatively simple compared to modern large language models, the nanochat model exhibits noticeably more stable and accurate behavior, demonstrating small-scale emergent abilities that are not reliably present in the picochat model.

- **Question 1:** Alice went to the market, then to the library. Where did Alice go first?
  - **Picochat:** Alice went to the market, then to the library. To start, Alice went to the library and asked the librarian, "What is the best place to go to?" The librarian replied, "The best place to go is the library."

    Alice went to the library and asked the librarian, "Where can I find the best books to read in the library?" The librarian replied, "The best place to find the best books to read in the library is the open book shelves on the main reading room."

    Alice then asked the librarian, "Where can I find the best books to read in the library?" The librarian replied, "The best books to read in the library is the open book shelves on the main reading room."

    Alice then asked the librarian, "Where can I find the best books to read in the library?" The librarian replied, "The best books to read in the library is the open book shelves on the main reading room."
  - **Nanochat:** To solve this problem, let's break it down into manageable parts. We know that Alice went to the market first and then to the library. The order of the trips doesn't matter, as we're looking for the first place she visited.

    Since Alice went to the market first, this would mean she went to the market first and then went to the library. However, this isn't the only place she might have visited.

    Given that Alice went to the market, we can't conclude where she went first. Therefore, the first place she visited could be anywhere between the market and the library.

    The correct order is that Alice went to the library first, and then she went to the market. This makes sense because the market would have been the first place she visited, and the library would have been the second place she visited.
- **Question 2:** Bob went to the park. He saw a dog and a cat. Who did he see first?
  - **Picochat:** Assistant: To solve this problem, let's break it down into manageable parts. We know that Bob saw a dog and a cat. We're also told that he saw a dog and a cat.

    Since the dog is the dog's companion, the dog must be the companion of Bob. This means the dog must be the companion of Bob, and the dog must be the companion of Bob, which is the dog's companion.

    Now, let's find the relationship between the dog's companion and Bob. We know that Bob saw a dog, and he saw a cat. If Bob saw a cat, then he must have saw a dog. If Bob saw a dog, then he must have saw a cat.

    Since Bob saw a dog and a cat, and Bob saw a cat, it logically follows that Bob saw a cat. Thus, Bob saw a cat.
  - **Nanochat:** Assistant: To solve this problem, let's break it down into its basic components. We know that Bob saw a dog and a cat. This information alone does not tell us who saw which animal first.

    The key to this problem is identifying the order in which the animals were seen. Since Bob saw a dog first, and then a cat, we can conclude that the dog was the first animal Bob saw. The cat

was the second animal Bob saw. However, this conclusion relies on the order in which Bob saw the animals, which is not specified.

Therefore, the order in which the animals were seen is: dog, cat.

- **Question 3:** Mary has 3 red shirts and 2 blue shirts. How many shirts does she have in total?
  - **Picochat:** To find the total number of shirts Mary has, we need to consider that she can only have either 3 shirts or 2 shirts. This means that Mary can have 3 + 2 = 5 shirts with 1 shirt each.

    Since Mary has 3 shirts in total, we can set up an equation based on this information. The total number of shirts is 5, which means Mary has 5 x 3 = 15 shirts.

    However, we still need to consider that she has 2 shirts. So, we need to add 2 shirts to the total number of shirts. This means Mary has 2 x 15 = 30 shirts.

    Therefore, Mary has a total of 15 shirts in total.
  - **Nanochat:** To find the total number of shirts Mary has, we need to consider the number of red shirts and blue shirts she has.

    Mary has 3 red shirts and 2 blue shirts, which is a total of 3 + 2 = 5 shirts.
- **Question 4:** If a basket has 4 apples and 5 oranges, how many pieces of fruit are there in total?
  - **Picochat:** To find the total number of pieces of fruit, we need to consider the proportion of apples and oranges that each fruit has.

    We know that 4 apples and 5 oranges have a total of 4 + 5 = 9 apples. Since there are 4 apples and 5 oranges, we can calculate the total number of pieces of fruit by multiplying the number of apples by the number of oranges: 4 apples * 9 oranges = 36 apples.

    However, we also need to consider that the number of oranges does not affect the total number of fruit. Therefore, the total number of pieces of fruit is 36 apples (or 4 apples and 5 oranges) divided by 4 apples, or 36/4 = 6 apples.

    So, there are 6 pieces of fruit in total.
  - **Nanochat:** To find the total number of pieces of fruit in the basket, we need to consider both the apples and oranges. We start by adding the number of apples to the number of oranges, which gives us 4 apples + 5 oranges = 9 pieces of fruit.

    However, this counts every piece of fruit twice, since we're considering the same apples and oranges multiple times. To find the total number of pieces, we need to divide the total number of apples by the number of oranges.

    Since there are 9 pieces of apples and 5 oranges, which is 1 more than a multiple of 2 (the number of oranges), we can divide 9 by 1 to find the total number of apples: 9 apples / 1 apple = 9 pieces of fruit.

    Therefore, the basket has 9 pieces of fruit.
- **Question 5:** Write a Python list comprehension that generates the squares of numbers from 1 to 5.
  - **Picochat:** "'python numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

    # Using list comprehension squares = [i**2 for i in range(1, 6)]

    # Using the range function squares = [i**2 for i in range(1, 6)]

    # Using the square function squares = [i**2 for i in range(1, 6)]

    # Using the map function squares = [i**2 for i in range(1, 6)]

    # Using the join function squares = list(map(lambda x: x**2, range(1, 6)))

    # Using the square function squares = list(map(lambda x: x**2, range(1, 6)))

    # Using square function squares = list(map(lambda x: x**2, range(1, 6)))

    # Using the map function squares = list(map(lambda x

- **Nanochat:** “‘python squares = [i ** 2 for i in range(1, 6)] “‘
  This list comprehension uses a for loop to iterate over the numbers 1 through 5, squares each number, and collects the results in a new list.
  Alternatively, you can use the ‘map‘ function in combination with a lambda function:
  “‘python squares = list(map(lambda x: x ** 2, range(1, 6))) “‘
  Both of these approaches will produce the same result, but the list comprehension is generally more concise and efficient.

- **Question 6:** What planet is known as the Red Planet?
  - **Picochat:** The Red Planet is a massive, dynamic body located in the outer reaches of the solar system, and is one of the most volcanically active bodies in the solar system. It is also known as a planet due to its unique atmospheric composition, which is thought to be similar to that of Earth, where the gravitational pull is so strong that it allows for the formation of very deep, dark volcanic rocks. However, unlike Earth, Red Planet is not a planet of the solar system, but rather a unique and dynamic system that has existed for billions of years.
  - **Nanochat:** The planet known as the Red Planet is Mars. Mars is the second largest planet in the solar system, and it is also the most Earth-like of the five planets in our solar system. It has a reddish hue due to its iron oxide content, which is a result of its rusty red composition. Mars is also home to numerous volcanoes, canyons, and polar ice caps, making it a fascinating celestial body. Its unique landscape and geological features have captivated scientists and explorers for centuries, and it continues to be an important target for scientific research and exploration.

- **Question 7:** Who wrote Romeo and Juliet?
  - **Picochat:** Romeo and Juliet is a classic tale of love, loss, and tragedy. It was written by the Italian playwright Giovanni Bellini, and it’s widely regarded as one of the most iconic and beloved love stories in history. According to the biography of the famous Italian playwright, Giovanni Bellini’s Romeo and Juliet was written by Bellini in 1597, and it was translated into French, but it’s not the only one.
    There are several other books and movies that have made the legend of Romeo and Juliet more widely known, including ”The Lion King”, ”La Rivière”, and ”Romeo and Juliet” by the French playwright Gerardus de Reeves. However, some of the most beloved stories in history include ”The Lion King”, ”The Montagues of Death”, and ”Hercules of the 3rd”. These stories have had a profound impact on various cultures, including the English, British, and French, and continue to inspire and captivate audiences around the world.
  - **Nanochat:** Romeo and Juliet was written by William Shakespeare, an English playwright, poet, and actor who is widely regarded as one of the most famous playwrights in history. He is best known for writing thirty-seven plays, including tragedies, comedies, and historical dramas, and is the author of many famous works, including Romeo and Juliet, A Midsummer Night’s Dream, Hamlet, and Macbeth.

- **Question 8:** Which ocean is the largest on Earth?
  - **Picochat:** The largest ocean on our planet is the North Atlantic Ocean, also known as the North Atlantic Ocean. However, the term ”largest” is somewhat misleading because it’s not the largest, but the largest, ocean in the world. The North Atlantic Ocean is the largest ocean in the world, reaching an area of approximately 1,700 square kilometers (1,600 square miles), or roughly 4 times the size of the United States. The largest ocean on Earth is the Great Barrier Reef in Australia, which is the largest coral reef on the planet.
  - **Nanochat:** The largest ocean on Earth is the Pacific Ocean. It covers about 63% of the Earth’s surface, making it the largest ocean in the world. Its size is comparable to the size of the largest lake on Earth, Lake Superior, which is the largest freshwater lake in the world. The Pacific Ocean has a total area of approximately 632 million square kilometers, making it the largest ocean on Earth.

- **Question 9:** What is the plural of "mouse"?
  - **Picochat:** The plural of "mouse" is "mouse" or "mouse" as in "the cat" or "the dog." However, in the context of the sentence, "mouse" refers to a person, so we can also use "the dog" or "the cat" for referring to a person.
  - **Nanochat:** The plural of "mouse" is "mice." This is because in English, the plural forms of nouns ending in "-e" or "-y" are usually formed by adding "s" or "es" to the end of the noun, rather than adding an "s" or "es" to the end of the noun itself. For example, "mice" becomes "mice" and "mice" becomes "mice" in the plural form.
- **Question 10:** How many continents are there on Earth?
  - **Picochat:** The Earth is divided into 365 distinct regions, each with its own unique geography and climate. These regions are separated by the North Pole, which is a major natural boundary between the Earth and the Sun.

    The North Pole is situated in the northern hemisphere, and it is the only point between the equator and the North Pole. The South Pole is located in the middle of the Earth's equator and is closer to the equator than to the North Pole. The North Pole is often seen as a kind of "triangle" between the equator and the North Pole, where the North Pole is the center of the Earth and the South Pole is the opposite point.

    The South Pole is also located in the northern hemisphere, but it is closer to the equator than to the North Pole. The South Pole is often seen as a kind of "climber" or "triangle" between the equator and the North Pole, where the South Pole is the center of the Earth and the North Pole is the opposite point.
  - **Nanochat:** There are 7 recognized continents on Earth. These continents are typically geographical features of the planet, while some of them also have smaller, unofficial territories or archipelagos. The continents are:

    Africa, Asia, Australia, Europe, North America, South America, and Antarctica. These six continents are the largest and most well-known, but there are many other smaller and larger continents as well.

References

Ainslie, Joshua et al. (2023). *GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints*. arXiv: 2305.13245 [cs.CL]. URL: https://arxiv.org/abs/2305.13245.

Chowdhery, Aakanksha et al. (2022). "PaLM: Scaling Language Modeling with Pathways". In: *arXiv preprint arXiv:2204.02311*. arXiv: 2204.02311 [cs.CL]. URL: https://arxiv.org/abs/2204.02311.

DeepSeek-AI et al. (2025). "DeepSeek-V3.2: Pushing the Frontier of Open Large Language Models". In: *arXiv preprint arXiv:2512.02556*. arXiv: 2512.02556 [cs.CL]. URL: https://arxiv.org/abs/2512.02556.

Kaplan, Jared et al. (2020). "Scaling Laws for Neural Language Models". In: *arXiv preprint arXiv:2001.08361*.

Karpathy, Andrej (2024). *Nanochat Training Discussion*. https://github.com/karpathy/nanochat/discussions/481. Accessed 2026.

Kim, Bo-Kyeong et al. (2024). "Shortened LLaMA: Depth Pruning for Large Language Models with Comparison of Retraining Methods". In: *arXiv preprint arXiv:2402.02834*. arXiv: 2402.02834 [cs.LG]. URL: https://arxiv.org/abs/2402.02834.

Press, Ofir, Noah A. Smith, and Mike Lewis (2021). "Shortformer: Better Language Modeling using Shorter Inputs". In: arXiv: 2012.15832 [cs.CL]. URL: https://arxiv.org/abs/2012.15832.

Qiu, Zihan et al. (2025). *Gated Attention for Large Language Models: Non-linearity, Sparsity, and Attention-Sink-Free*. arXiv: 2505.06708 [cs.CL]. URL: https://arxiv.org/abs/2505.06708.

Shazeer, Noam (2020). "GLU Variants Improve Transformer". In: *arXiv preprint arXiv:2002.05202*. arXiv: 2002.05202 [cs.LG]. URL: https://arxiv.org/abs/2002.05202.

Vries, Harm de (Sept. 2023). *In the long (context) run*. https://www.harmdevries.com/post/context-length/. Personal blog.

Ye, Tianzhu et al. (2025). *Differential Transformer*. arXiv: 2410.05258 [cs.CL]. URL: https://arxiv.org/abs/2410.05258.